UNIT III: Constructors, Destructors and Operator Overloading

Section A: Debugging Questions

Question 1: Copy Constructor Pass-by-Value Infinite Recursion Trap

Topic: Copy Constructors, Parameter Passing Mechanisms

Corrected Code:

```
#include <iostream>
#include <cstring>

class DynamicString {
private:
    char *buffer;
    int length;

public:
    // Parameterized constructor
    DynamicString(const char *str) {
        length = std::strlen(str);
        buffer = new char[length + 1];
        std::strcpy(buffer, str);
    }

    // Correct deep copy constructor
    DynamicString(const DynamicString &source) {
        length = source.length;
        buffer = new char[length + 1];
        std::strcpy(buffer, source.buffer);
    }

    // Destructor
    ~DynamicString() {
        delete[] buffer;
    }

    void print() const {
```

```
        std::cout << buffer << std::endl;
    }
};

int main() {
    DynamicString s1("Antigravity C++");

    DynamicString s2 = s1;

    std::cout << "Original String: ";
    s1.print();

    std::cout << "Copied String: ";
    s2.print();

    return 0;
}
```

Output:

Original String: Antigravity C++

Copied String: Antigravity C++

Question 2: Shallow Copy & Double-Free Runtime Crash

Topic: Dynamic Constructors, Destructors, Shallow vs. Deep Copy

Corrected Code:

```
#include <iostream>

class AudioBuffer {
public:
    int *samples;
    int capacity;

    AudioBuffer(int cap) : capacity(cap) {
        samples = new int[capacity];

        for (int i = 0; i < capacity; ++i) {
            samples[i] = i * 10;
        }
```

```
        std::cout << "Original buffer allocated"
                  << std::endl;
    }

    // Deep copy constructor
    AudioBuffer(const AudioBuffer &other) {
        capacity = other.capacity;
        samples = new int[capacity];

        for (int i = 0; i < capacity; ++i) {
            samples[i] = other.samples[i];
        }

        std::cout << "Copied buffer allocated"
                  << std::endl;
    }

    ~AudioBuffer() {
        std::cout << "Buffer deallocated"
                  << std::endl;

        delete[] samples;
    }
};

void processAudio(AudioBuffer buf) {
    std::cout << "Processing sample 0: "
              << buf.samples[0] << std::endl;
}

int main() {
    AudioBuffer track(5);

    processAudio(track);

    std::cout << "Back in main, sample 0: "
              << track.samples[0] << std::endl;

    return 0;
```

```
}
```

Output: Original buffer allocated

Copied buffer allocated

Processing sample 0: 0

Buffer deallocated

Back in main, sample 0: 0

Buffer deallocated

Question 3: Stream Insertion Operator `<<` Member Function Violation

Topic: Defining Operator Overloading, Member vs. Non-Member/Friend

Corrected Code:

```
#include <iostream>
using namespace std;

class Fraction {
private:
    int numerator;
    int denominator;

public:
    Fraction(int n, int d)
        : numerator(n), denominator(d) {}

    // Friend non-member stream insertion operator
    friend ostream& operator<<(ostream &os,
                               const Fraction &f);
};

// Operator definition outside the class
ostream& operator<<(ostream &os, const Fraction &f) {
    os << f.numerator << "/" << f.denominator;
    return os;
}

int main() {
    Fraction f1(3, 4);
```

```
    cout << "Fraction: " << f1 << endl;

    return 0;
}
```

Output: Fraction: ¾

Question 4: Prefix vs. Postfix Increment Signature Inversion

Topic: Overloading Unary Operators (Prefix `++`, Postfix `++(int)`)

Corrected Code:

```
#include <iostream>
using namespace std;

class StepCounter {
private:
    int steps;

public:
    StepCounter(int s = 0) : steps(s) {}

    // Prefix increment
    StepCounter& operator++() {
        ++steps;
        return *this;
    }

    // Postfix increment
    StepCounter operator++(int) {
        StepCounter temp = *this;
        steps++;
        return temp;
    }

    void display() const {
        cout << "Steps: " << steps << endl;
    }
```

```
};

int main() {
    StepCounter c1(10);

    cout << "Prefix Increment:" << endl;

    StepCounter c2 = ++c1;

    cout << "c1: ";
    c1.display();

    cout << "c2: ";
    c2.display();

    cout << "\nPostfix Increment:" << endl;

    StepCounter c3 = c1++;

    cout << "c1: ";
    c1.display();

    cout << "c3: ";
    c3.display();

    return 0;
}
```

Output:

```
Prefix Increment:
c1: Steps: 11
c2: Steps: 11
Postfix Increment:
c1: Steps: 12
c3: Steps: 11
```

Question 5: Violation of Fundamental Operator Overloading Rules

Topic: Rules for Overloading Operators

Corrected Code:

```
#include <iostream>
using namespace std;

class MathVector {
public:
    double x, y;

    MathVector(double a = 0, double b = 0)
        : x(a), y(b) {}

    // Overload equality operator
    bool operator==(const MathVector &other) const {
        return (x == other.x && y == other.y);
    }

    // Overload addition operator
    MathVector operator+(const MathVector &other) const {
        return MathVector(x + other.x,
                          y + other.y);
    }

    // Exponentiation using a member function
    MathVector power(int exponent) const {
        MathVector result(1, 1);

        for (int i = 0; i < exponent; ++i) {
            result.x *= x;
            result.y *= y;
        }

        return result;
    }
```

```
    void display() const {
        cout << "(" << x << ", " << y << ")" << endl;
    }
};

int main() {
    MathVector v1(2, 3);
    MathVector v2(2, 3);

    MathVector sum = v1 + v2;

    cout << "Vector 1: ";
    v1.display();

    cout << "Vector 2: ";
    v2.display();

    cout << "Addition: ";
    sum.display();

    if (v1 == v2) {
        cout << "Vectors are Equal" << endl;
    } else {
        cout << "Vectors are Not Equal" << endl;
    }

    MathVector result = v1.power(2);

    cout << "Power 2: ";
    result.display();

    return 0;
}
```

Output:

```
Vector 1: (2, 3)
```

Vector 2: (2, 3)

Addition: (4, 6)

Vectors are Equal

Power 2: (4, 9)

Section B: Pseudo-Code Writing Questions

Question 1: Dynamic String Engine with Deep Copy & Overloaded `+`

Topic: Dynamic Constructors, Deep Copy Constructors, Destructors, Binary `+`

pseudo-code:

```
CLASS DynStr
PRIVATE:
    dataPtr : POINTER TO CHARACTER
    charCount : INTEGER
PUBLIC:
    CONSTRUCTOR(sourceString : STRING)
        charCount ← LENGTH(sourceString)
        dataPtr ← ALLOCATE ARRAY[charCount + 1]
        FOR i ← 0 TO charCount - 1
            dataPtr[i] ← sourceString[i]
        ENDFOR
        dataPtr[charCount] ← '\0'
    END CONSTRUCTOR
    COPY CONSTRUCTOR(REF other AS DynStr)
        charCount ← other.charCount
        dataPtr ← ALLOCATE ARRAY[charCount + 1]
        FOR i ← 0 TO charCount - 1
            dataPtr[i] ← other.dataPtr[i]
        ENDFOR
        dataPtr[charCount] ← '\0'
    END COPY CONSTRUCTOR
    OPERATOR +(REF rhs AS DynStr) RETURNS DynStr
        DECLARE newLength : INTEGER
        DECLARE tempBuffer : POINTER TO CHARACTER
        newLength ← charCount + rhs.charCount
        tempBuffer ← ALLOCATE ARRAY[newLength + 1]
        FOR i ← 0 TO charCount - 1
            tempBuffer[i] ← dataPtr[i]
        ENDFOR
```

```
        FOR j ← 0 TO rhs.charCount - 1
            tempBuffer[charCount + j] ← rhs.dataPtr[j]
        ENDFOR
        tempBuffer[newLength] ← '\0'
        DECLARE result : DynStr
        result ← CONSTRUCT DynStr FROM tempBuffer
        DEALLOCATE tempBuffer
        RETURN result
    END OPERATOR
    METHOD Display()
        DISPLAY dataPtr
    END METHOD
    DESTRUCTOR()
        DEALLOCATE dataPtr
        dataPtr ← NULL
    END DESTRUCTOR
ENDCLASS
BEGIN MAIN
    DECLARE strA : DynStr
    DECLARE strB : DynStr
    DECLARE strC : DynStr
    strA ← CONSTRUCT DynStr("Hello ")
    strB ← CONSTRUCT DynStr("World!")
    strC ← strA + strB
    DISPLAY "String A: "
    strA.Display()
    DISPLAY "String B: "
    strB.Display()
    DISPLAY "Concatenated String: "
    strC.Display()
END MAIN
```

output:

```
String A: Hello
String B: World!
Concatenated String: Hello World!
```

Question 2: Complex Number System with Constructor Overloading & Operators (`+`, `*`, `==`)

Topic: Constructor Overloading, Operator Overloading (`+`, `*`, `==`)

pseudo-code:

```
CLASS Complex
PRIVATE:
```

```
    real : REAL
    imag : REAL
PUBLIC:
    CONSTRUCTOR()
        real ← 0.0
        imag ← 0.0
    END CONSTRUCTOR
    CONSTRUCTOR(r : REAL)
        real ← r
        imag ← 0.0
    END CONSTRUCTOR
    CONSTRUCTOR(r : REAL, i : REAL)
        real ← r
        imag ← i
    END CONSTRUCTOR
    OPERATOR +(REF other AS Complex) RETURNS Complex
        DECLARE result : Complex
        result.real ← real + other.real
        result.imag ← imag + other.imag
        RETURN result
    END OPERATOR
    OPERATOR *(REF other AS Complex) RETURNS Complex
        DECLARE result : Complex
        result.real ←
            (real * other.real) -
            (imag * other.imag)
        result.imag ←
            (real * other.imag) +
            (imag * other.real)
        RETURN result
    END OPERATOR
    OPERATOR ==(REF other AS Complex) RETURNS BOOLEAN
        IF real = other.real AND imag = other.imag THEN
            RETURN TRUE
        ELSE
            RETURN FALSE
        ENDIF
    END OPERATOR
    METHOD Display()
        DISPLAY real, " + ", imag, "i"
    END METHOD
ENDCLASS
```

```
BEGIN MAIN
    DECLARE C1, C2, C3 : Complex
    C1 ← CONSTRUCT Complex(3, 2)
    C2 ← CONSTRUCT Complex(1, 4)
    C3 ← C1 * C2
    DISPLAY "Complex Number 1:"
    C1.Display()
    DISPLAY "Complex Number 2:"
    C2.Display()
    DISPLAY "Product:"
    C3.Display()
    IF C1 == C2 THEN
        DISPLAY "Numbers are Equal"
    ELSE
        DISPLAY "Numbers are Not Equal"
    ENDIF
END MAIN
```

output:

```
Complex Number 1:
3 + 2i
Complex Number 2:
1 + 4i
Product:
-5 + 14i
Numbers are Not Equal
```

Question 3: Financial Currency Class with Stream I/O & Unary Negation

Topic: Unary Operator Overloading, Friend Stream Operators (`<<`, `>>`)

pseudo-code:

```
CLASS Currency
PRIVATE:
    dollars : INTEGER
    cents : INTEGER
PUBLIC:
    CONSTRUCTOR(d : INTEGER = 0, c : INTEGER = 0)
        DECLARE totalCents : INTEGER
        totalCents ← (d * 100) + c
        dollars ← totalCents DIV 100
        cents ← ABS(totalCents MOD 100)
        IF totalCents < 0 AND cents <> 0 THEN
```

```
        dollars ← dollars - 1
        cents ← 100 - cents
    ENDIF
END CONSTRUCTOR
OPERATOR -() RETURNS Currency
    DECLARE totalCents : INTEGER
    totalCents ← (dollars * 100) + cents
    totalCents ← -totalCents
    DECLARE result : Currency
    result ← CONSTRUCT Currency(
        totalCents DIV 100,
        totalCents MOD 100
    )
    RETURN result
END OPERATOR
FRIEND OPERATOR <<(
    REF outStream,
    REF cur AS Currency
)
    DISPLAY "$"
    DISPLAY cur.dollars
    DISPLAY "."
    DISPLAY TWO_DIGIT_FORMAT(cur.cents)
END OPERATOR
FRIEND OPERATOR >>(
    REF inStream,
    REF cur AS Currency
)
    INPUT cur.dollars
    INPUT cur.cents
    DECLARE totalCents : INTEGER
    totalCents ←
        (cur.dollars * 100) + cur.cents
    cur.dollars ← totalCents DIV 100
    cur.cents ← ABS(totalCents MOD 100)
    IF totalCents < 0 AND cur.cents <> 0 THEN
        cur.dollars ← cur.dollars - 1
        cur.cents ← 100 - cur.cents
```

```
        ENDIF
    END OPERATOR
ENDCLASS
BEGIN MAIN
    DECLARE cur, negativeCur : Currency
    DISPLAY "Enter dollars and cents:"
    INPUT cur
    DISPLAY "Normalized Currency: ", cur
    negativeCur ← -cur
    DISPLAY "After Unary Negation: ", negativeCur
END MAIN
```

output:

```
Enter dollars and cents:
5 275
Normalized Currency: $7.75
After Unary Negation: $-7.75
```

Question 4: Polynomial Expression with Overloaded Subscript `[]` & Destructor

Topic: Dynamic Constructors, Destructors, Overloaded Subscript Operator `[]`

pseudo-code:

```
CLASS Polynomial
PRIVATE:
    degree : INTEGER
    coeffArray : POINTER TO REAL
PUBLIC:
    CONSTRUCTOR(deg : INTEGER)
        degree ← deg
        coeffArray ← ALLOCATE ARRAY[degree + 1]
        FOR i ← 0 TO degree
            coeffArray[i] ← 0.0
        ENDFOR
    END CONSTRUCTOR
    COPY CONSTRUCTOR(REF other AS Polynomial)
        degree ← other.degree
        coeffArray ← ALLOCATE ARRAY[degree + 1]
        FOR i ← 0 TO degree
            coeffArray[i] ← other.coeffArray[i]
        ENDFOR
    END COPY CONSTRUCTOR
    OPERATOR [](index : INTEGER) RETURNS REF REAL
```

```
    IF index >= 0 AND index <= degree THEN
        RETURN REF coeffArray[index]
    ELSE
        DISPLAY "ERROR: Invalid coefficient index"
        STOP
    ENDIF
END OPERATOR
OPERATOR +(REF other AS Polynomial)
    RETURNS Polynomial
    DECLARE maxDegree : INTEGER
    DECLARE result : Polynomial
    maxDegree ← MAX(degree, other.degree)
    result ← CONSTRUCT Polynomial(maxDegree)
    FOR i ← 0 TO maxDegree
        DECLARE firstCoeff, secondCoeff : REAL
        firstCoeff ← 0.0
        secondCoeff ← 0.0
        IF i <= degree THEN
            firstCoeff ← coeffArray[i]
        ENDIF
        IF i <= other.degree THEN
            secondCoeff ← other.coeffArray[i]
        ENDIF
        result.coeffArray[i] ←
            firstCoeff + secondCoeff
    ENDFOR
    RETURN result
END OPERATOR
METHOD Display()
    FOR i ← degree DOWNTO 0 STEP -1
        IF coeffArray[i] <> 0 THEN
            DISPLAY coeffArray[i]
            IF i > 0 THEN
                DISPLAY "x^", i
            ENDIF
            IF i > 0 THEN
                DISPLAY " + "
            ENDIF
        ENDIF
    ENDFOR
END METHOD
DESTRUCTOR()
```

```
        DEALLOCATE coeffArray
        coeffArray ← NULL
    END DESTRUCTOR
ENDCLASS
BEGIN MAIN
    DECLARE P1, P2, P3 : Polynomial
    // P1(x) = 3x^2 + 2x + 5
    P1 ← CONSTRUCT Polynomial(2)
    P1[0] ← 5
    P1[1] ← 2
    P1[2] ← 3
    // P2(x) = 4x^3 + x^2 + 7
    P2 ← CONSTRUCT Polynomial(3)
    P2[0] ← 7
    P2[1] ← 0
    P2[2] ← 1
    P2[3] ← 4
    P3 ← P1 + P2
    DISPLAY "Polynomial P1:"
    P1.Display()
    DISPLAY "Polynomial P2:"
    P2.Display()
    DISPLAY "Sum Polynomial:"
    P3.Display()
END MAIN
```

output:

```
Polynomial P1:
3x^2 + 2x + 5

Polynomial P2:
4x^3 + 1x^2 + 7

Sum Polynomial:
4x^3 + 4x^2 + 2x + 12
```

Question 5: 2D Vector with Prefix/Postfix Increment & Compound Assignment

Topic: Unary Operators (Prefix vs Postfix `++`), Compound Assignment (`+=`)

pseudo-code:

```
CLASS Vector2D
PRIVATE:
    x : REAL
    y : REAL
PUBLIC:
    CONSTRUCTOR(a : REAL = 0.0, b : REAL = 0.0)
```

```
 x ← a
        y ← b
    END CONSTRUCTOR
    OPERATOR ++() RETURNS REF Vector2D
        x ← x + 1.0
        y ← y + 1.0
        RETURN REF THIS
    END OPERATOR
    OPERATOR ++(dummy : INTEGER) RETURNS Vector2D
        DECLARE oldValue : Vector2D
        oldValue ← COPY OF THIS
        x ← x + 1.0
        y ← y + 1.0
        RETURN oldValue
    END OPERATOR
    OPERATOR +=(REF other AS Vector2D)
        RETURNS REF Vector2D
        x ← x + other.x
        y ← y + other.y
        RETURN REF THIS
    END OPERATOR
    METHOD Display()
        DISPLAY "(", x, ", ", y, ")"
    END METHOD
ENDCLASS
BEGIN MAIN
    DECLARE v1, v2, v3 : Vector2D
    v1 ← CONSTRUCT Vector2D(5.0, 8.0)
    // Prefix increment
    v2 ← ++v1
    DISPLAY "After Prefix Increment:"
    DISPLAY "v1 = "
    v1.Display()
    DISPLAY "v2 = "
    v2.Display()
    // Postfix increment
    v3 ← v1++
    DISPLAY "After Postfix Increment:"
```

```
    DISPLAY "v1 = "
    v1.Display()
    DISPLAY "v3 = "
    v3.Display()
    // Compound assignment demonstration
    DECLARE v4 : Vector2D
    v4 ← CONSTRUCT Vector2D(2.0, 3.0)
    v1 += v4
    DISPLAY "After Compound Assignment:"
    v1.Display()
END MAIN
```

output:

```
After Prefix Increment:
v1 = (6, 9)
v2 = (6, 9)
After Postfix Increment:
v1 = (7, 10)
v3 = (6, 9)
After Compound Assignment:
(9, 13)
```